

### Esercizio A1 – Complessità – Punti $\geq 8$

Dal codice, è possibile vedere che la matrice di input è quadrata. Consideriamo, per semplicità, il caso in cui  $n$  è una potenza di 2. Il risultato è valido anche per dimensioni che non rispettano tale vincolo.

Il codice calcola la somma di tutti i valori, in tempo  $O(n^2)$ , e poi chiama ricorsivamente se stessa su due porzioni di dimensione  $n/2$ , che corrispondono al secondo e quarto quadrante se interpretiamo la matrice come un piano cartesiano. Il caso base si verifica quando  $x_1 = x_2$  e  $y_1 = y_2$ ; in questo caso, la somma dei valori richiede tempo  $O(1)$  (essendoci un valore solo da sommare) e non vengono effettuate chiamate ricorsive.

La funzione di complessità è quindi:

$$T(n) = \begin{cases} 1 & n \leq 1 \\ 2T(n/2) + n^2 & n > 1 \end{cases}$$

Applicando il Master Theorem, otteniamo  $\alpha = \log_2 2 = 1$  e  $\beta = 2$ . Poiché  $\beta > \alpha$ , per il Master Theorem  $T(n) = \Theta(n^2)$ .

### Esercizio A2 – Diretto o no? – Punti $\geq 10$

È possibile risolvere il problema in molti modi diversi, con complessità quali  $O(mn)$ ,  $O(n^2)$ ,  $O(m+n)$ .

La soluzione più banale consiste nel verificare, per ogni arco  $(u, v)$  di  $G$ , se  $u$  è nel vettore/lista di adiacenza di  $v$ . In un grafo completo, questo algoritmo ha complessità  $O(mn)$ , perché per ognuno degli  $m$  archi è necessario effettuare una ricerca nel vettore/lista di adiacenza del nodo destinazione, con complessità  $O(n)$ .

---

```
boolean isUndirected(GRAPH G)
```

---

```
  foreach  $x \in G.V()$  do
    foreach  $y \in G.adj(x)$  do
      if  $x \notin G.adj(y)$  then
        return false
  return true
```

---

Una soluzione leggermente più efficiente consiste nel costruire una matrice di adiacenza a partire dalle liste di adiacenza e poi verificare se la matrice è simmetrica, con costo  $O(n^2)$ .

---

```
boolean isUndirected(GRAPH G)
```

---

```
  boolean[] M = new int[1...G.n][1...G.n] = {false}
  foreach  $x \in G.V()$  do
    foreach  $y \in G.adj(x)$  do
      M[x][y] = true
  for  $x = 1$  to  $n - 1$  do
    for  $y = x + 1$  to  $n$  do
      if  $M[x][y] \neq M[y][x]$  then
        return false
  return true
```

---

Il problema della prima soluzione è che sebbene `adj()` restituisce un insieme, normalmente diamo per scontato che questo insieme sia implementato come una lista/vettore non ordinato. Questo significa che l'operatore  $\in$  abbia costo  $O(n)$ . Se si assume che l'insieme sia implementato in modo più efficiente (ma generalmente spreca più memoria), si possono ottenere complessità migliori.

Per la regola che è lecito utilizzare le "API" fornite dal libro e dalle slide, ma se volete che si comportino diversamente da quanto descritto nel libro, dovete re-implementarle, proponiamo una soluzione che crea un vettore di insiemi basati su tabella hash, uno per ogni nodo  $x$ , che contengono i nodi vicini di  $x$ . In questo modo, l'operatore  $\in$  ha costo  $O(1)$  e il costo totale è  $O(m+n)$ , perché l'istruzione `if` viene eseguita  $O(m)$  volte e inizializzazione e scorrimento dei nodi costano  $O(n)$ .

---

**boolean** isUndirected(**GRAPH**  $G$ )

---

```
SET[] neighbors = new SET[1...G.n]
for x = 1 to n do
  neighbors[x] = Set()
foreach x ∈ G.V() do
  foreach y ∈ G.adj(x) do
    neighbors[x].insert(y)
foreach x ∈ G.V() do
  foreach y ∈ G.adj(x) do
    if x ∉ neighbors[y] then
      return false
return true
```

---

### Esercizio A3 – Prossimità – Punti $\geq 12$

La soluzione proposta sfrutta il meccanismo di calcolo della posizione nel vettore degli alberi binari heap. L'idea è che la radice si trova in posizione 1; i figli sinistro e destro del nodo  $i$ -esimo si trovano in posizione  $2i$  e  $2i + 1$ . La funzione principale richiama la funzione ricorsiva `locate()`, che cerca un nodo nell'albero e restituisce il suo livello e la sua posizione nell'albero heap.

Se i due nodi hanno lo stesso livello, si calcola la differenza di posizione; visto che si chiede di calcolare il numero di nodi intermedi nel livello, si sottrae 1. Se i due nodi non appartengono allo stesso livello, si ritorna  $+\infty$ .

---

**proximity**(**TREE**  $T$ , **TREE**  $t_1$ , **TREE**  $t_2$ )

---

```
int level1, pos1 = locate( $T$ ,  $t_1$ , 0, 1)
int level2, pos2 = locate( $T$ ,  $t_2$ , 0, 1)
if level1 == level2 then
  return |pos2 - pos1| - 1
return  $+\infty$ 
```

---

La funzione `locate()` prende in input la radice  $t$  del sottoalbero che si sta esaminando, il nodo  $x$  che si sta cercando, il livello  $level$  della radice e la posizione  $pos$  della radice nel vettore heap. La funzione restituisce due valori: il livello e la posizione del nodo, se è stato trovato, (-1, -1) se non è stato trovato. La parte ricorsiva esplora i due sottorami sinistro e destro. Se il nodo viene trovato in uno dei due rami, si restituisce il livello corrente e la posizione. Il calcolo del livello e della posizione viene effettuato nel passaggio dei parametri, passando  $level + 1$  e  $pos \cdot 2$  oppure  $pos \cdot 2 + 1$ .

---

**(int, int)** locate(**TREE**  $t$ , **TREE**  $x$ , **int**  $level$ , **int**  $pos$ )

---

```
if t == nil then
  return (-1, -1)
else if t == x then
  return (level, pos)
else
  int, int level, pos = locate( $t.left$ ,  $x$ , level + 1, pos · 2)
  if level ≥ 0 then
    return (level, pos)
  int, int level, pos = locate( $t.right$ ,  $x$ , level + 1, pos · 2 + 1)
  if level ≥ 0 then
    return (level, pos)
  return (-1, -1)
```

---

La complessità è ovviamente  $O(n)$ , in quanto si tratta di una visita in profondità.

**Casi particolari** Ci sarebbero tantissimi casi particolari da distinguere, che sono stati evitati dall'assunzione che entrambi  $t_1$  e  $t_2$  appartengano all'albero e siano distinti. Ne parliamo qui per illustrare cosa succede, questa discussione non era richiesta dall'esercizio.

- Se  $t_1$  e  $t_2$  sono lo stesso nodo, viene restituito -1, perché il livello è uguale e la differenza di posizione è 0;
- Se uno dei due nodi non appartiene all'albero e l'altro sì, viene restituito  $+\infty$ , perché uno ha livello -1 e l'altro un livello positivo o nullo;
- Se entrambi i nodi non appartengono all'albero, viene restituito -1